`Docker` Challenge

Task 1 — Create a Custom Docker Image and Push to Docker Hub

1. Objective

To create a customized Docker image using a Dockerfile, run it as a container on an AWS EC2 Ubuntu server, test the application, and push the image to Docker Hub.

2. What is this task about?

In this task, we created a simple website called Vineeth Dance Studio and packaged it into a Docker image.

Instead of installing and configuring a web server manually every time, we used Docker to package the website with Nginx. We then ran the image as a container and uploaded it to Docker Hub so it could be downloaded and used on other servers.

3. Purpose of the task

- Learn how to install and use Docker on an AWS EC2 instance.
- Understand how to create a Dockerfile.
- Learn to build a custom Docker image.
- Run a Docker container and access a website through a browser.
- Upload an image to Docker Hub for sharing and reuse.

4. Tools used

Tool	Purpose
AWS EC2	Provides the Ubuntu virtual server
Docker	Builds and runs containers

Dockerfile	Defines how to create the image
Nginx	Serves the website
Docker Hub	Stores and shares the Docker image

5. Procedure

Step 1 — Launch an EC2 instance

Created an Ubuntu EC2 instance in AWS to use as the Docker server.

Purpose: Provides a virtual machine where Docker can be installed and the website can run.

Step 2 — Install Docker

Updated the package list and installed Docker.

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
```

Purpose: Installs Docker and starts the Docker service.

Step 3 — Create the project directory

```
mkdir ~/vineeth-dance-studio
cd ~/vineeth-dance-studio
```

Purpose: Creates a separate directory to store the website and Dockerfile.

Step 4 — Create the website

Created an `index.html` file containing the Vineeth Dance Studio website, including information about dance classes such as Hip Hop, Bollywood, and Freestyle.

Purpose: Provides the custom content that will be served by Nginx.

Step 5 — Create the Dockerfile

Created a file named `Dockerfile` with the following content:

```
FROM nginx:alpine
```

COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80

Explanation:

- `FROM nginx:alpine` — Uses the lightweight Nginx image as the base.
- `COPY` — Copies our custom website into Nginx's web directory.
- `EXPOSE 80` — Documents the port used by Nginx inside the container.

Purpose: Defines the instructions for creating our custom Docker image.

Step 6 — Build the Docker image

sudo docker build -t vineeth-dance-studio:v1 .

Purpose: Builds a Docker image containing the Nginx web server and our custom website.

Step 7 — Run the Docker container

We initially ran the container using port 81:

sudo docker run -d --name vineeth-dance-container -p 81:80 vineeth-dance-studio:v1

Later, to use port 8081, we changed the container mapping to:

sudo docker stop vineeth-dance-container
sudo docker rm vineeth-dance-container
sudo docker run -d --name vineeth-dance-container -p 8081:80 vineeth-dance-studio:v1

Purpose: Runs the website inside a container and maps EC2 port 8081 to container port 80.

To verify the container:

sudo docker ps

Expected port mapping:

0.0.0.0:8081->80/tcp

The EC2 security group must allow inbound TCP port 8081 to access the website externally.

Step 8 — Push the image to Docker Hub

Tagged the image with the Docker Hub repository name:

sudo docker tag vineeth-dance-studio:v1 vineethgupta/vineeth-dance-studio:v1

Logged in to Docker Hub using a personal access token, then pushed the image:

sudo docker push vineethgupta/vineeth-dance-studio:v1

Purpose: Stores the custom image in a remote repository so it can be pulled and used on other machines.

Docker Hub repository: [vineethgupta/vineeth-dance-studio]

6. Real-time example

Imagine a dance academy wants to publish its website.

Without Docker, a developer might need to install and configure Nginx manually on every new server.

With Docker:

1. The developer creates the website and Dockerfile.
2. Docker builds the website into an image.
3. The image is pushed to Docker Hub.
4. Another server downloads the image and starts a container.
5. The website runs using the same packaged configuration.

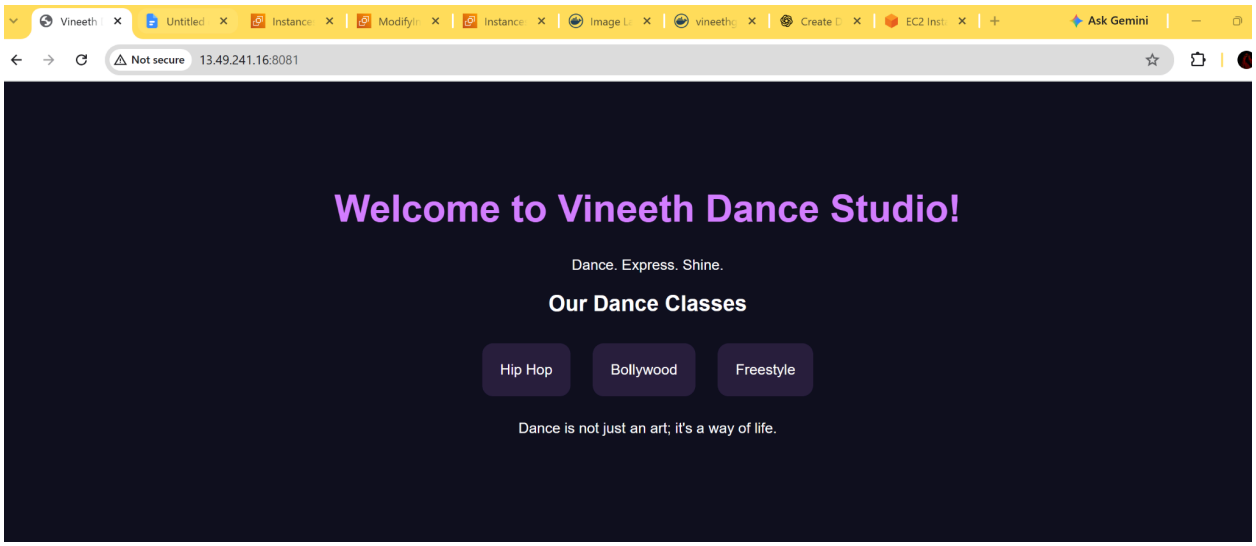
This approach is useful for deploying websites, APIs, and applications consistently across development, testing, and production environments.

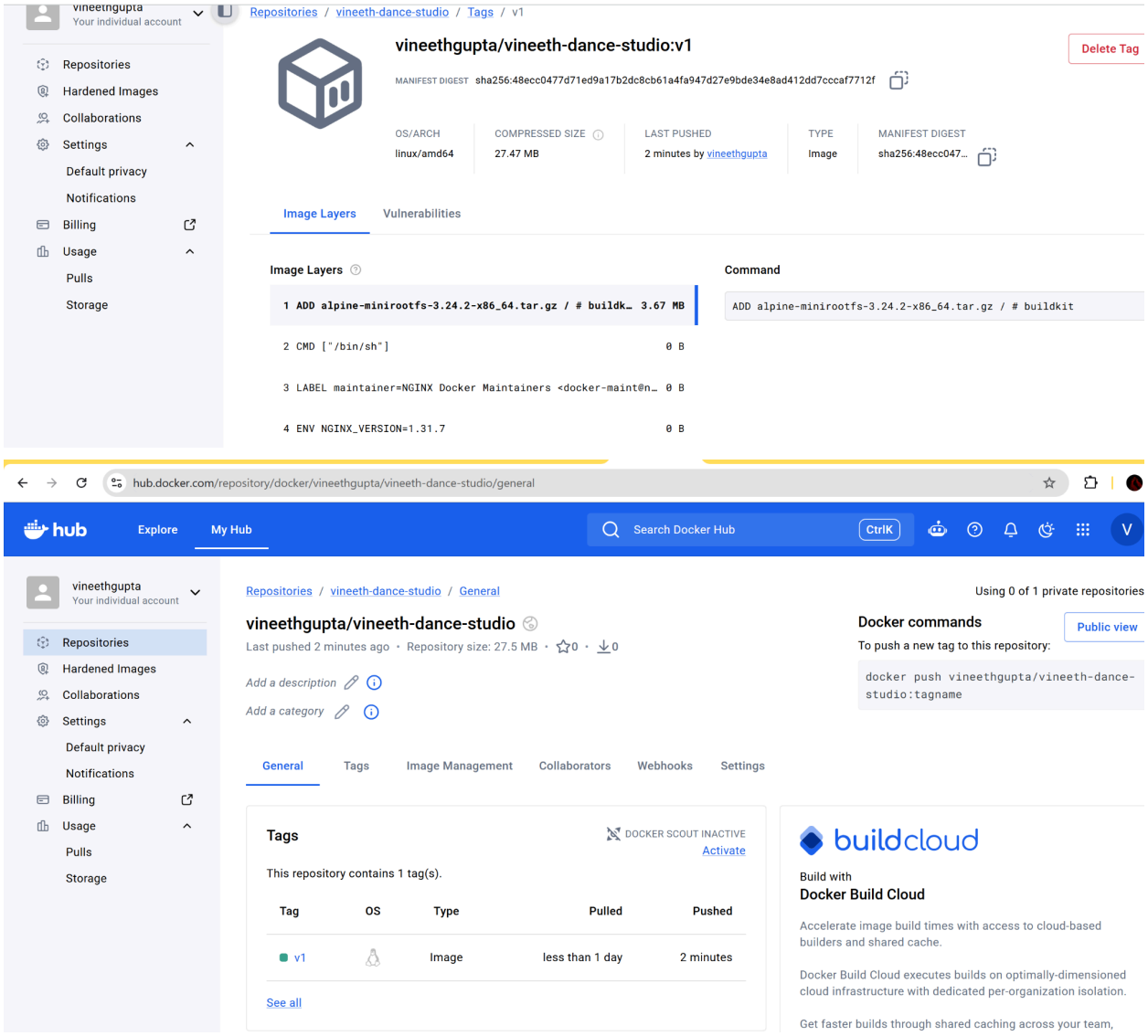
7. Conclusion

Successfully created a custom Docker image for Vineeth Dance Studio, ran it as a container on an AWS EC2 Ubuntu instance, and pushed the image to Docker Hub.

This task provided practical experience with Dockerfiles, image building, container execution, port mapping, and image repositories.

Key learning: A Docker image is the packaged application, a container is the running instance of that image, and Docker Hub is a remote place to store and share images.





Task 2 — Push a Docker Image to Amazon ECR

1. Objective

To push an existing custom Docker image from an AWS EC2 Ubuntu instance to Amazon Elastic Container Registry (ECR) and verify that the image is stored successfully.

2. What is this task about?

In Task 1, we created a custom website image called `vineeth-dance-studio:v1` and uploaded it to Docker Hub.

In Task 2, we uploaded the same image to Amazon ECR, a private container image registry provided by AWS. This allows us to store Docker images within AWS and use them for application deployment.

3. Purpose

- Learn how to use Amazon ECR.
- Understand how EC2 IAM roles provide AWS permissions.
- Authenticate Docker with a private ECR registry.
- Tag and push Docker images to ECR.
- Verify uploaded images in the AWS Console.

4. Tools used

Tool	Purpose
AWS EC2	Runs the Docker commands
AWS CLI	Communicates with AWS services
IAM role	Grants EC2 permission to access ECR
Docker	Tags and uploads the image
Amazon ECR	Stores the Docker image

5. Step-by-step procedure

Step 1 — Create an ECR repository

1. Open the Amazon ECR Console.
2. Select the AWS region `eu-north-1`.
3. Click Create repository.
4. Choose Private repository.
5. Enter the repository name:

vineeth-dance-studio

6. Keep the default settings and click Create repository.

Purpose: Creates a private location in AWS to store the Docker image.

Step 2 — Install AWS CLI

On the EC2 Ubuntu terminal, run:

```
sudo apt update
sudo apt install awscli -y
```

Verify the installation:

```
aws --version
```

Purpose: AWS CLI allows us to execute AWS commands from the EC2 terminal.

Step 3 — Create and attach an IAM role

1. Open the AWS IAM Console.
2. Create a role named `EC2-ECR-Push-Role` for the EC2 service.
3. Attach the AWS managed policy `AmazonEC2ContainerRegistryPowerUser`.
4. Open the EC2 Console and select your instance.
5. Choose Actions → Security → Modify IAM role.
6. Attach the role to your instance.

Purpose: Gives EC2 the necessary permissions to upload images to Amazon ECR without storing permanent AWS access keys on the server.

Step 4 — Verify AWS identity

Run:

```
aws sts get-caller-identity
```

Expected result includes your AWS account ID:

063298174631

Purpose: Confirms that the EC2 instance can authenticate with AWS using its attached IAM role.

Step 5 — Log in to Amazon ECR

Run:

```
aws ecr get-login-password --region eu-north-1 | sudo docker login --username AWS --password-stdin 063298174631.dkr.ecr.eu-north-1.amazonaws.com
```

Expected output:

Login Succeeded

Purpose: Authenticates Docker with your private ECR registry so it can upload images.

Step 6 — Tag the Docker image

Run:

sudo docker tag vineeth-dance-studio:v1 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth-dance-studio:v1

Verify the image:

sudo docker images

Purpose: Adds the ECR repository address to the existing image. This does not rebuild the image.

Step 7 — Push the image to ECR

Run:

sudo docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth-dance-studio:v1

Wait for the upload to finish.

Purpose: Uploads the custom Docker image to Amazon ECR.

Step 8 — Verify the uploaded image

1. Open the Amazon ECR Console.
2. Click the `vineeth-dance-studio` repository.
3. Check that the image tag `v1` is displayed.

Your image URI is:

063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth-dance-studio:v1

6. Real-time example

Imagine a company develops an application using Docker and wants to deploy it on AWS.

The developer builds the Docker image and pushes it to Amazon ECR. An EC2 server, ECS service, or another authorized deployment environment can then pull that image from ECR and run the application.

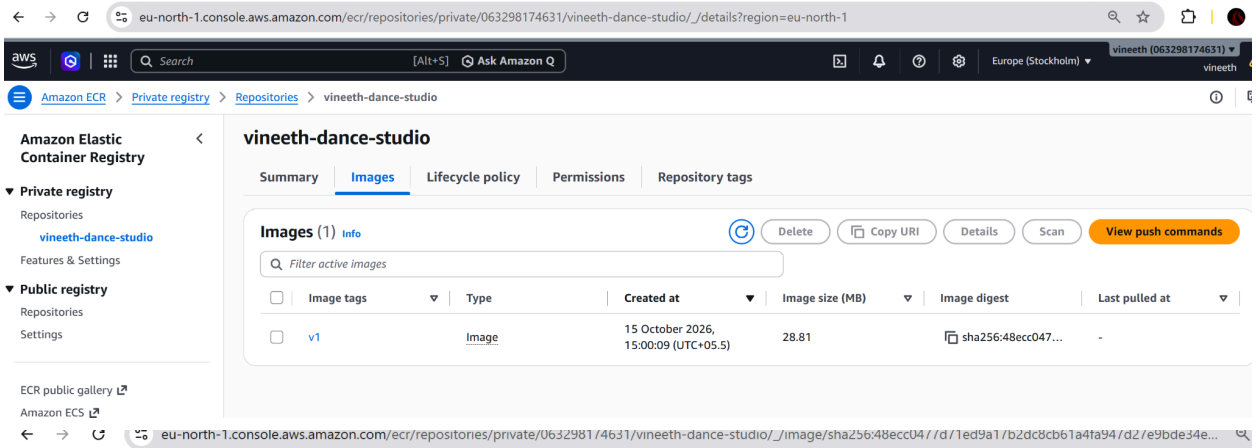
Real-world benefit: ECR provides a centralized private registry for storing and managing application images within AWS.

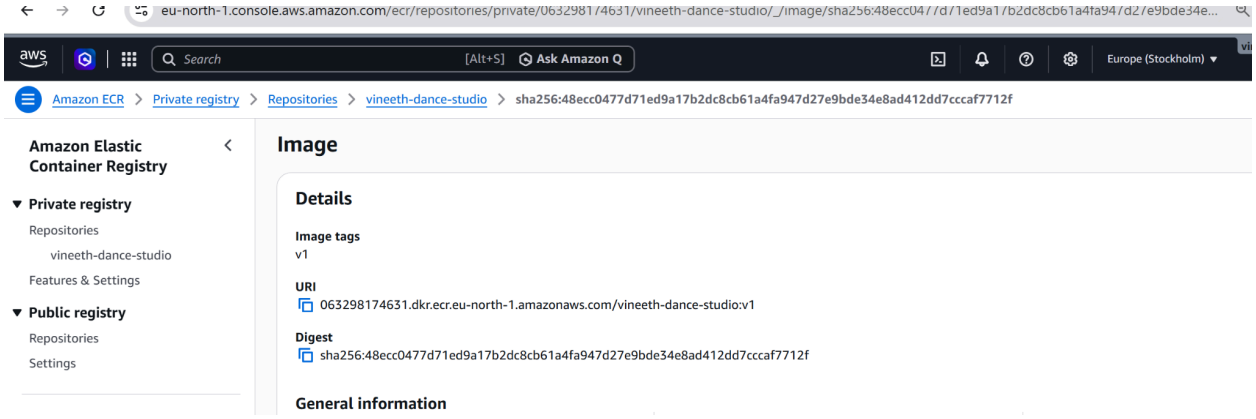
7. Conclusion

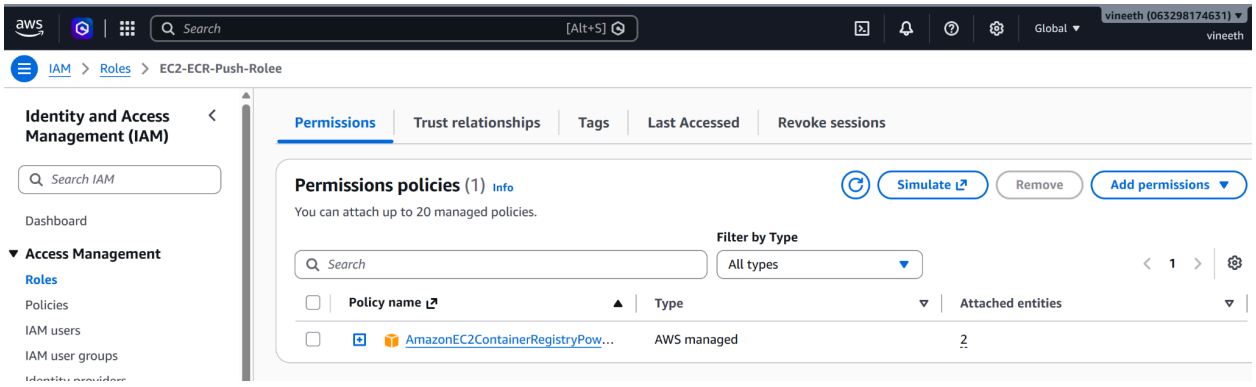
Successfully created a private Amazon ECR repository, configured AWS CLI and EC2 IAM permissions, authenticated Docker with ECR, tagged the existing custom image, and pushed it to Amazon ECR.

The uploaded image was verified in the AWS Console with the tag v1.

Key learning: Docker builds and runs images, while Amazon ECR stores Docker images so they can be securely accessed by authorized AWS services.







Task 3: Push a Docker Image to Amazon ECR

Objective

To push the existing Docker image from an Ubuntu EC2 instance to Amazon Elastic Container Registry (ECR).

Step 1: Create an ECR Repository

1. Open the AWS Management Console.
2. Search for **Amazon ECR**.
3. Select **Repositories** → **Create repository**.
4. Enter the repository name: `vineeth-dance-studio`.
5. Select **Private** and click **Create repository**.

Step 2: Build the Docker Image

Navigate to the project directory:

cd ~/vineeth-dance-studio

Build the image if it has not already been built:

sudo docker build -t vineeth-dance-studio:v1 .

Step 3: Log in to Amazon ECR

Run the following command:

aws ecr get-login-password --region eu-north-1 | sudo docker login --username AWS --password-stdin 063298174631.dkr.ecr.eu-north-1.amazonaws.com

Expected output:

Login Succeeded

Step 4: Tag the Docker Image

sudo docker tag vineeth-dance-studio:v1 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth-dance-studio:v1

Step 5: Push the Image to ECR

sudo docker push 063298174631.dkr.ecr.eu-north-1.amazonaws.com/vineeth-dance-studio:v1

Expected output:

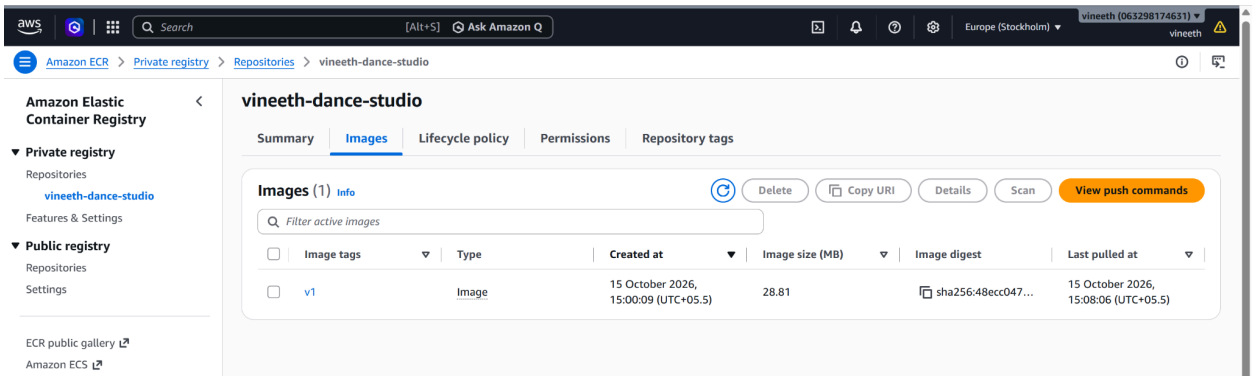
v1: digest: sha256:...

Step 6: Verify the Image

1. Open AWS Console → Amazon ECR.
2. Select **Repositories**.
3. Open `vineeth-dance-studio`.
4. Click the **Images** tab.
5. Confirm that the image tag `v1` is displayed.

Result

Successfully pushed the Docker image `vineeth-dance-studio:v1` to Amazon ECR and verified its presence in the AWS Console.



Task 4.Provision an EC2 Instance Using Terraform and Install Jenkins

Objective

To create an AWS EC2 instance using Terraform and install Jenkins on the Ubuntu server.

Step 1: Prepare the Ubuntu machine

Connect to your Ubuntu EC2 machine where Terraform is installed.

Check Terraform:

terraform --version

Check AWS credentials:

aws sts get-caller-identity

Purpose: Verify that Terraform and AWS CLI are available and AWS authentication is working.

Step 2: Create a Terraform directory

mkdir -p ~/terraform-jenkins
cd ~/terraform-jenkins

Create the configuration file:

nano main.tf

Paste this Terraform configuration:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "eu-north-1"
}

resource "aws_security_group" "jenkins_sg" {
  name        = "terraform-jenkins-sg"
  description = "Allow SSH and Jenkins access"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Jenkins"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

data "aws_ami" "ubuntu" {
```

```
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_instance" "jenkins" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  tags = {
    Name = "Terraform-Jenkins-Server"
  }
}

output "jenkins_public_ip" {
  value = aws_instance.jenkins.public_ip
}
```

Save the file using `Ctrl+O`, press Enter, and exit with `Ctrl+X`.

Step 3: Initialize Terraform

terraform init

Purpose: Downloads the AWS provider required by Terraform.

Step 4: Validate the configuration

terraform validate

Expected result:

Success! The configuration is valid.

Step 5: Review the execution plan

terraform plan

Purpose: Shows the infrastructure changes Terraform intends to make before applying them.

Step 6: Provision the EC2 instance

terraform apply

Type `yes` when prompted.

In your completed lab, Terraform reported:

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

This was because the EC2 instance had already been created and Terraform only needed to save the output value.

Step 7: Get the EC2 public IP

terraform output jenkins_public_ip

Your completed lab returned:

13.48.43.222

Step 8: Connect to the EC2 instance

Open AWS Console → EC2 → Instances.

1. Select `Terraform-Jenkins-Server`.
2. Click Connect.
3. Choose EC2 Instance Connect.
4. Click Connect.

Step 9: Install Java

Run these commands inside the Jenkins EC2 terminal:

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre wget
```

Purpose: Jenkins requires Java to run.

Step 10: Add the Jenkins repository

Download the current signing key:

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

Add the repository:

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
| sudo tee /etc/apt/sources.list.d/jenkins.list
```

Update the package list:

```
sudo apt update
```

Step 11: Install and start Jenkins

Install:

```
sudo apt install -y jenkins
```

Enable and start Jenkins:

```
sudo systemctl enable --now jenkins
```

Check the service:

```
sudo systemctl status jenkins --no-pager
```

Expected status:

Active: active (running)

Step 12: Open Jenkins in the browser

Open:

http://13.48.43.222:8080

If the page does not open, check that inbound TCP port `8080` is allowed in the EC2 security group.

Step 13: Complete the Jenkins setup

Get the initial administrator password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

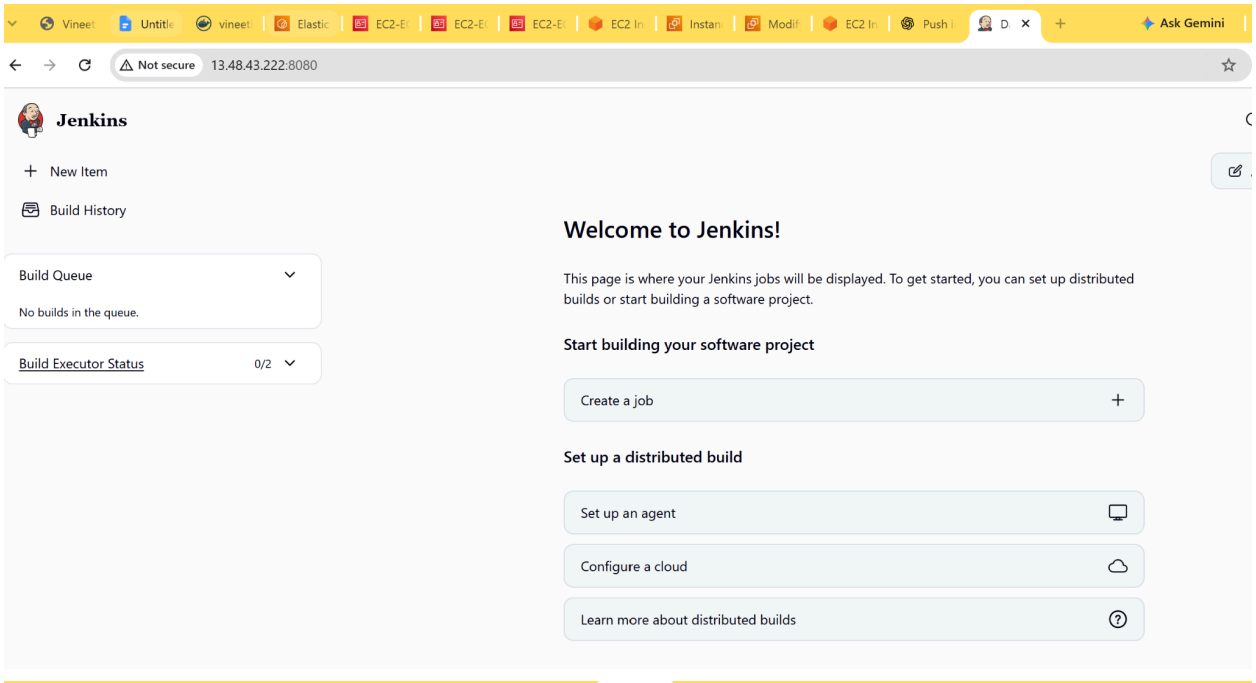
Then:

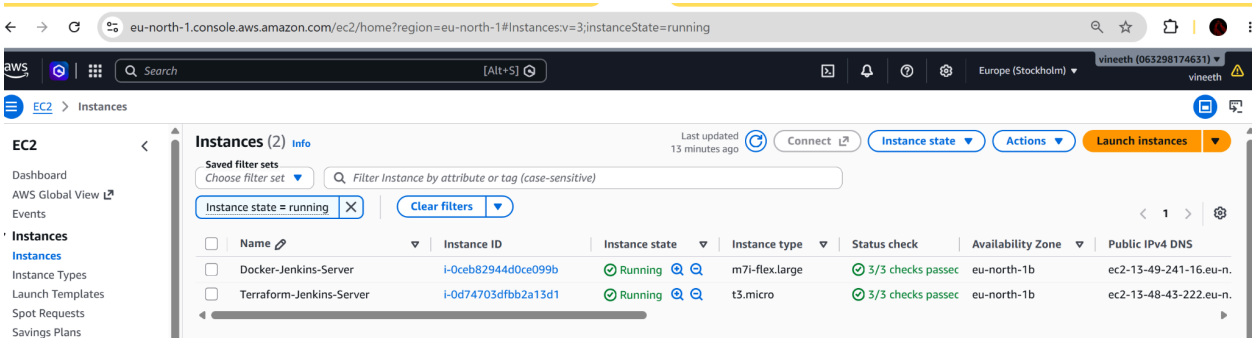
1. Enter the password in the Jenkins setup page.
2. Click Install suggested plugins.
3. Create your administrator user.
4. Save and finish the setup.
5. Open the Jenkins dashboard.

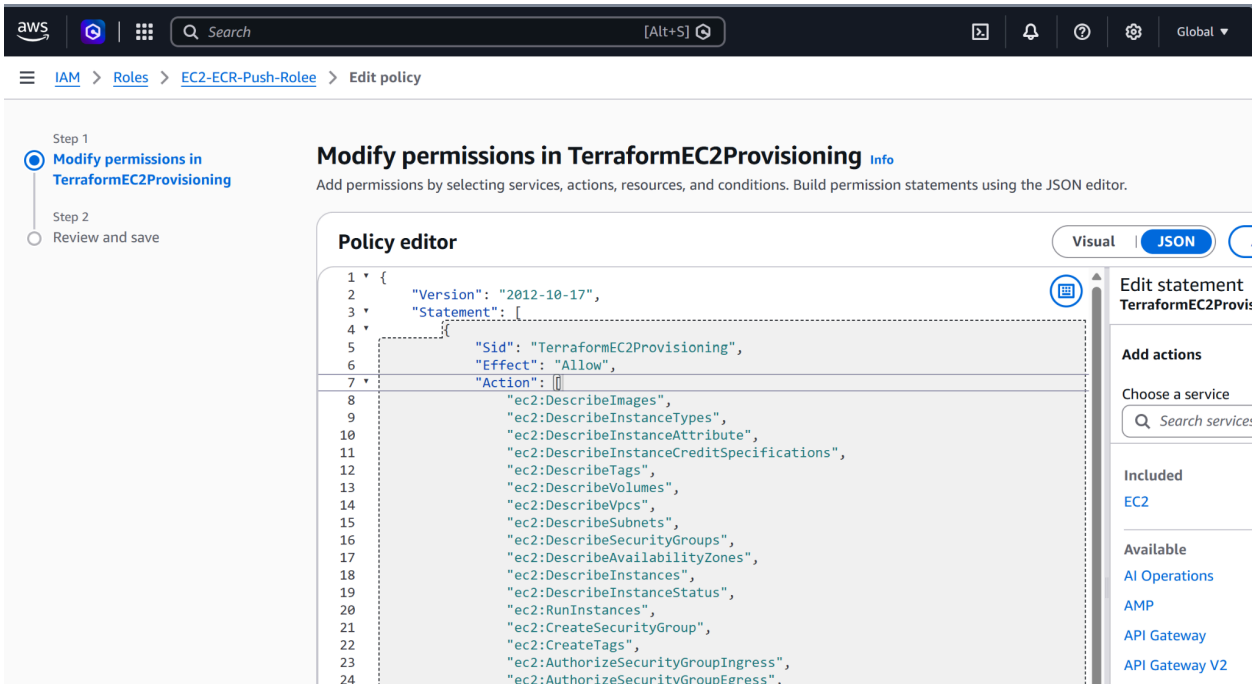
Result

Successfully provisioned an AWS EC2 instance using Terraform and installed and configured Jenkins on Ubuntu.

Important for future runs: This lab's EC2 instance already exists. Running `terraform apply` with the same configuration should not create another instance. The current security group allows SSH and Jenkins access from anywhere; for better security, restrict SSH to your own public IP.







Task 5: Create one Jenkins job to build and push the Docker image to Docker Hub.

Practical DevOps lab · Step-by-step notes · Real-time example

1. Objective

To create a Jenkins Pipeline job that automatically checks out Python application source code from GitHub, builds a Docker image, tests the build process, logs in to Docker Hub securely, and pushes the image to the Docker Hub repository.

2. Purpose

In real-world software development, developers frequently update application code. Instead of manually building and uploading a Docker image every time, Jenkins automates these steps.

For example, when a developer updates a Python application, Jenkins can build a new Docker image and push it to Docker Hub. The operations or deployment team can then pull that image and run it on a server.

3. Tools used

Tool	Purpose

AWS EC2 (Ubuntu)	Provides the Linux server
Terraform	Provisions the EC2 instance
Jenkins	Automates the pipeline
GitHub	Stores application source code
Docker	Builds and runs the application container
Docker Hub	Stores and distributes Docker images

Docker Hub image: `vineethgupta/python-app:v1`

Jenkins server

`http://13.48.43.222:8080`

Application port

`8080`

Docker Hub credential ID

`dockerhub-creds`

5. Architecture

Developer commits code

GitHub repository

Jenkins Pipeline

Checkout → Build → Login → Push

Docker image

vineethgupta/python-app:v1

Docker Hub

Image stored and available to pull

6. Complete practical steps performed

Step 1 — Provision the EC2 instance using Terraform

Purpose: Create a Linux server to host Jenkins and Docker.

You completed this in your previous task.

- Cloud provider: AWS
- Operating system: Ubuntu
- Region: `eu-north-1`
- Jenkins port: `8080`
- Terraform resource: EC2 instance

Terraform automates infrastructure creation, so you don't have to create the EC2 instance manually each time.

Step 2 — Install and verify Docker

Purpose: Docker is required to build images and run containers.

On your Ubuntu EC2 instance, check Docker:

```
docker --version
sudo docker ps
```

If Docker is installed but your user lacks permission, use `sudo docker` for manual commands. For Jenkins, ensure the Jenkins build user can access Docker.

Step 3 — Clone the Python source code

Purpose: Download the application from GitHub.

```
cd /tmp
git clone https://github.com/betawins/Python-app.git
cd /tmp/Python-app
ls -la
```

The repository contained:

- `app.py` — Python Flask application
- `requirements.txt` — Python dependencies
- `Dockerfile` — Docker image instructions
- `Jenkinsfile` — existing pipeline definition
- `templates/` — HTML templates

Step 4 — Create the Dockerfile

Purpose: Specify how Docker should package the application.

The original Dockerfile used Python 3.6. You replaced it with:

```
FROM python:3.11-slim

WORKDIR /opt

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["python", "app.py"]
```

Explanation:

Instruction	Purpose
`FROM`	Selects the base Python image
`WORKDIR`	Sets the application working directory
`COPY`	Copies application files into the image
`RUN`	Installs Python dependencies
`EXPOSE`	Documents the application port
`CMD`	Starts the Flask application

Step 5 — Build the Docker image

Purpose: Package the application and its dependencies into an image.

```
cd /tmp/Python-app
sudo docker build -t pythonapp:v1 .
```

You received a successful build:

```
Successfully built 68cfbfe0d8e0
Successfully tagged pythonapp:v1
```

Verify:

```
sudo docker images
```

The `pythonapp:v1` image was approximately 211 MB.

Step 6 — Run and test the container

Purpose: Confirm that the application works before publishing it.

Run:

```
sudo docker run -d \
  --name pythonapp-test \
  -p 8081:8080 \
  pythonapp:v1
```

Check the container:

```
sudo docker ps
```

Test the application:

```
curl http://localhost:8081
```

Your test returned HTML containing:

```
<title>Hello from Flask</title>
```

This confirmed that the Flask application was responding.

The port mapping means requests to port `8081` on the EC2 host are forwarded to port `8080` inside the container.

Step 7 — Configure Docker Hub credentials in Jenkins

Purpose: Allow Jenkins to authenticate securely with Docker Hub.

You already had this credential configured:

Setting	Value
Credential ID	`dockerhub-creds`
Username	`vineethgupta`
Credential type	Username with password
Description	Docker Hub credentials

Jenkins stores the secret, so the password does not need to be written directly in the pipeline script.

Step 8 — Create the Jenkins Pipeline job

Purpose: Automate the build and push process.

In Jenkins:

1. Click New Item.
2. Enter `Python-DockerHub-Pipeline`.
3. Select Pipeline.
4. Click OK.
5. Under Pipeline, select Pipeline script.
6. Paste the following pipeline.
7. Click Save.

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'vineethgupta/python-app'
        DOCKER_TAG = 'v1'
    }

    stages {
        stage('Checkout Source') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/Python-app.git'
            }
        }
```

```
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t DOCKER_IMAGE:{DOCKER_TAG} .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-creds',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh '''
                        echo "DOCKER_PASS"|dockerlogin − u"DOCKER_USER" --password-stdin
                    ''
                }
            }
        }

        stage('Push Docker Image') {
            steps {
                sh 'docker push DOCKER_IMAGE:{DOCKER_TAG}'
            }
        }
    }

    post {
        always {
            sh 'docker logout || true'
        }
    }
}
```

This pipeline performs four main operations:

1. Checkout Source: Downloads the application code from GitHub.
2. Build Docker Image: Creates `vineethgupta/python-app:v1`.
3. Login to Docker Hub: Authenticates using the Jenkins credential.
4. Push Docker Image: Uploads the image to Docker Hub.

Jenkins Pipeline supports building images and publishing them to a registry; Docker's documentation also explains the image tagging and push process.



Jenkins
+1

Step 9 — Execute the pipeline

Purpose: Run the automation and verify that every stage succeeds.

1. Open `Python-DockerHub-Pipeline`.
2. Click Build Now.
3. Open the build number.
4. Select Console Output.
5. Confirm that the stages completed successfully.

Your result: SUCCESS.

The Jenkins job successfully built and pushed the image.

Step 10 — Verify the image in Docker Hub

Purpose: Confirm that the image is available in the remote registry.

Open Docker Hub and sign in as `vineethgupta`.

Open the `python-app` repository and check the Tags tab for `v1`.

The image name is:

vineethgupta/python-app:v1

You can also test the published image on another Docker-enabled server:

docker pull vineethgupta/python-app:v1

```
docker run -d \
  --name pythonapp-remote-test \
  -p 8081:8080 \
  vineethgupta/python-app:v1
```

Then test the app:

curl http://localhost:8081

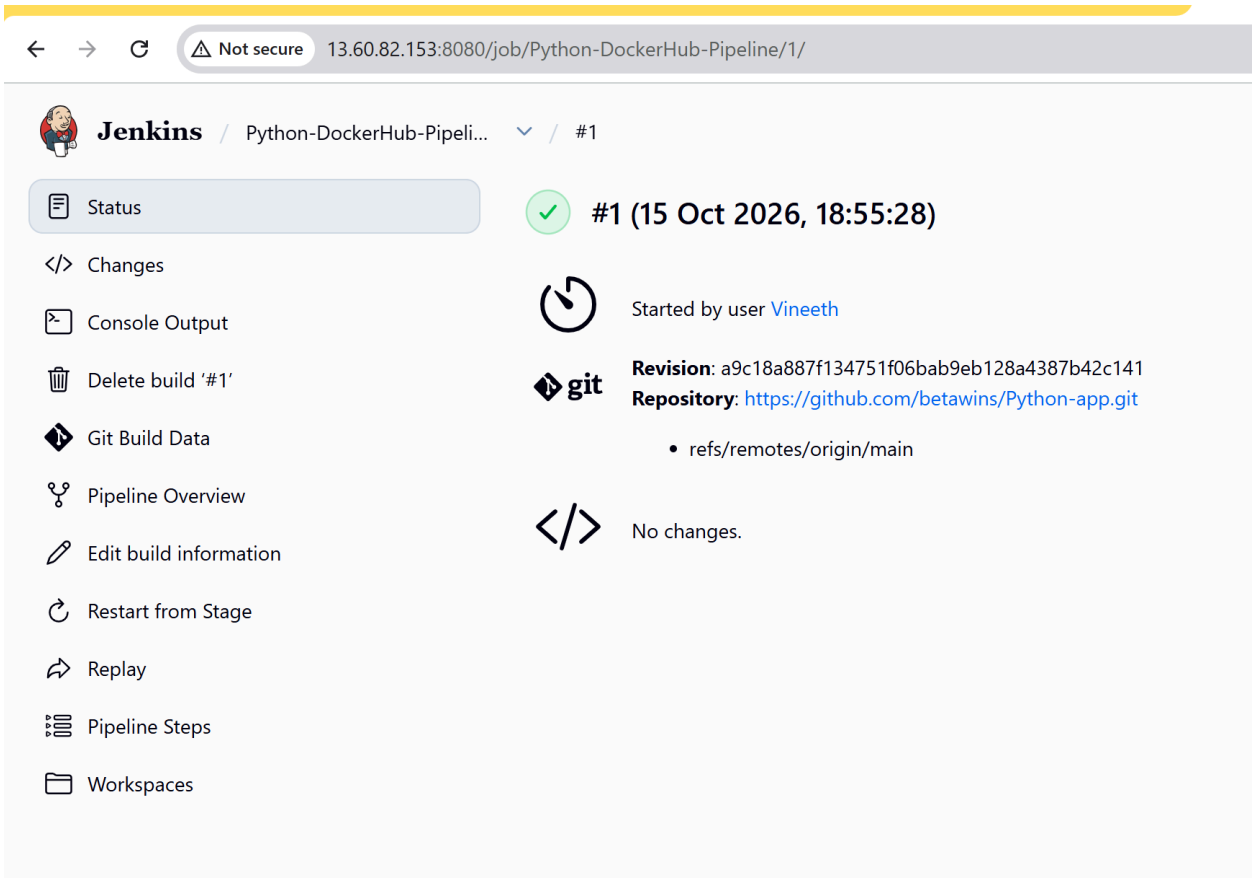
If the output returns the application's HTML, the published image works on that server too. The port `8081` must be available.

9. Conclusion

The Python Flask application was successfully containerized, tested locally, and published to Docker Hub through a Jenkins Pipeline.

This task demonstrated how GitHub, Jenkins, Docker, Docker Hub, and AWS EC2 work together in a basic CI/CD workflow. The key benefit is automation: Jenkins can repeatedly build and publish application images, reducing manual work and making application releases more consistent.

Result: The Python Docker Hub pipeline is complete. The remaining assignment tasks are still to be performed.



Task 6.From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Source code: https://github.com/betawins/docker-tasks.git

Objective

To containerize a frontend website using Docker, run the website in a container, verify that it works, and push the Docker image to Docker Hub.

Step 1 — Clone the source code

cd /tmp

git clone https://github.com/betawins/docker-tasks.git

cd /tmp/docker-tasks/Frontend_based_source

Purpose: Download the frontend source code from GitHub.

Step 2 — Create a Dockerfile

Create a file named `Dockerfile`:

FROM nginx:alpine

COPY . /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Purpose: Use Nginx to serve the HTML, CSS, and JavaScript files.

Step 3 — Build the Docker image

sudo docker build -t vineethgupta/frontend-app:v1 .

Purpose: Package the frontend website into a Docker image.

Verify:

sudo docker images

Step 4 — Run the Docker container

sudo docker run -d \

  --name frontend-app \

  -p 8082:80 \

  vineethgupta/frontend-app:v1

Purpose: Run the website in a Docker container and map EC2 port `8082` to container port `80`.

Step 5 — Test the application

sudo docker ps

curl -I http://localhost:8082

Expected result: An HTTP response such as `200 OK`.

Purpose: Verify that the frontend website is running.

Step 6 — Log in to Docker Hub

sudo docker login -u vineethgupta

Enter your Docker Hub access token when prompted.

Purpose: Authenticate with Docker Hub before uploading the image.

Step 7 — Push the Docker image

sudo docker push vineethgupta/frontend-app:v1

Purpose: Upload the Docker image to Docker Hub so it can be pulled and run on other servers.

Step 8 — Verify the image

Open Docker Hub — frontend-app and verify the `v1` tag.

You can also pull the image on another Docker-enabled server:

sudo docker pull vineethgupta/frontend-app:v1

Result

Successfully created, built, and ran the frontend Docker image and pushed it to Docker Hub.

Image name: `vineethgupta/frontend-app:v1`

Conclusion

The frontend website was containerized using Nginx, tested locally, and uploaded to Docker Hub. This allows the same application image to be distributed and deployed on other Docker-enabled servers without manually copying the application files.

```
i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-47-134:~$ sudo docker push vineethgupta/frontend-app:v1
The push refers to repository [docker.io/vineethgupta/frontend-app]
70683d58fa5c: Mounted from library/nginx
6638b436592c: Mounted from library/nginx
1fdd0614a47e: Mounted from library/nginx
e2de96513ba9: Mounted from library/nginx
6711e0c4f5a7: Mounted from library/nginx
b44956913298: Mounted from library/nginx
710ec83ca6cd: Mounted from library/nginx
3bf3671a4e6a: Mounted from library/nginx
06bb99cda917: Pushed
v1: digest: sha256:298a5ccbc28d8d8c0bba9cc2d020300a1c871f3eb1430902cfb335a552ef7f44 size: 2262
ubuntu@ip-172-31-47-134:~$
```

Task 7. “From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.”

1. Objective

To build a Java Spring Boot application using Maven, create a Docker image, run the application in a container, and push the image to Docker Hub.

2. Steps performed

Step 1 — Clone the source code

cd /tmp

git clone https://github.com/betawins/docker-tasks.git

Purpose: Download the source code from GitHub.

Step 2 — Navigate to the Java project

cd /tmp/docker-tasks/Java_based_source

ls

Purpose: Access the Java application files and `pom.xml`.

Step 3 — Install Maven

sudo apt-get update

sudo apt-get install -y maven

Purpose: Install Maven to compile and package the Java application.

Step 4 — Build the Java application

mvn -B -ntp clean package -DskipTests

Verify the generated JAR:

ls -lh target/

Generated file:

`target/spring-boot-web.jar`

Purpose: Compile the Java source code and package it into an executable JAR file.

Step 5 — Create the Dockerfile

cat > Dockerfile <<'EOF'

FROM eclipse-temurin:11-jre

WORKDIR /app

COPY target/spring-boot-web.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]

EOF

Check the Dockerfile:

cat Dockerfile

Purpose: Define the Java runtime, copy the JAR into the image, and specify how to start the application.

Step 6 — Build the Docker image

sudo docker build -t vineethgupta/java-app:v1 .

Verify:

sudo docker images | grep java-app

Image created:

`vineethgupta/java-app:v1`

Purpose: Package the Java application and its runtime into a Docker image.

Step 7 — Run the Docker container

sudo docker run -d --name java-app -p 8083:8080 vineethgupta/java-app:v1

Verify the container:

sudo docker ps

Purpose: Run the Java application inside a Docker container. Host port `8083` maps to application port `8080`.

Step 8 — Test the application

curl -I http://localhost:8083

Expected result:

HTTP/1.1 200

Browser URL:

`http://13.60.47.140:8083`

Purpose: Confirm that the Java application is responding to HTTP requests. The EC2 security group must allow inbound TCP port `8083` for external browser access.

Step 9 — Log in to Docker Hub

sudo docker login -u vineethgupta

Enter your Docker Hub access token when prompted.

Expected result:

Login Succeeded

Purpose: Authenticate with Docker Hub before pushing the image.

Step 10 — Push the image to Docker Hub

sudo docker push vineethgupta/java-app:v1

Verify it on Docker Hub:

https://hub.docker.com/r/vineethgupta/java-app

Purpose: Upload the Docker image to the registry so it can be pulled and deployed on other servers.

3. Result

Successfully completed

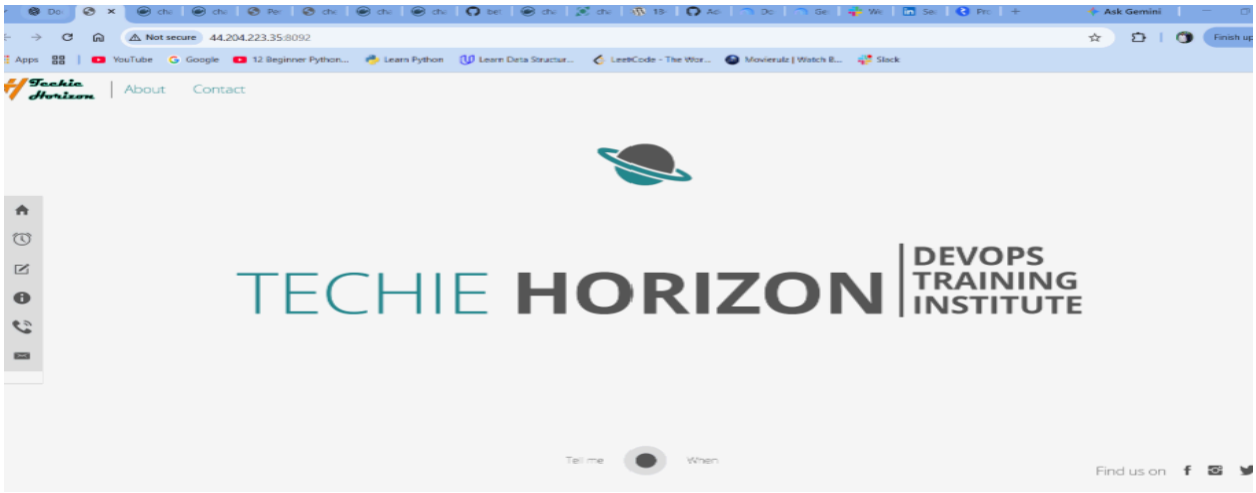
- Java application built successfully using Maven.
- Dockerfile created.
- Docker image built: `vineethgupta/java-app:v1`
- Container started and HTTP test returned `200 OK`.
- Docker image pushed successfully to Docker Hub.

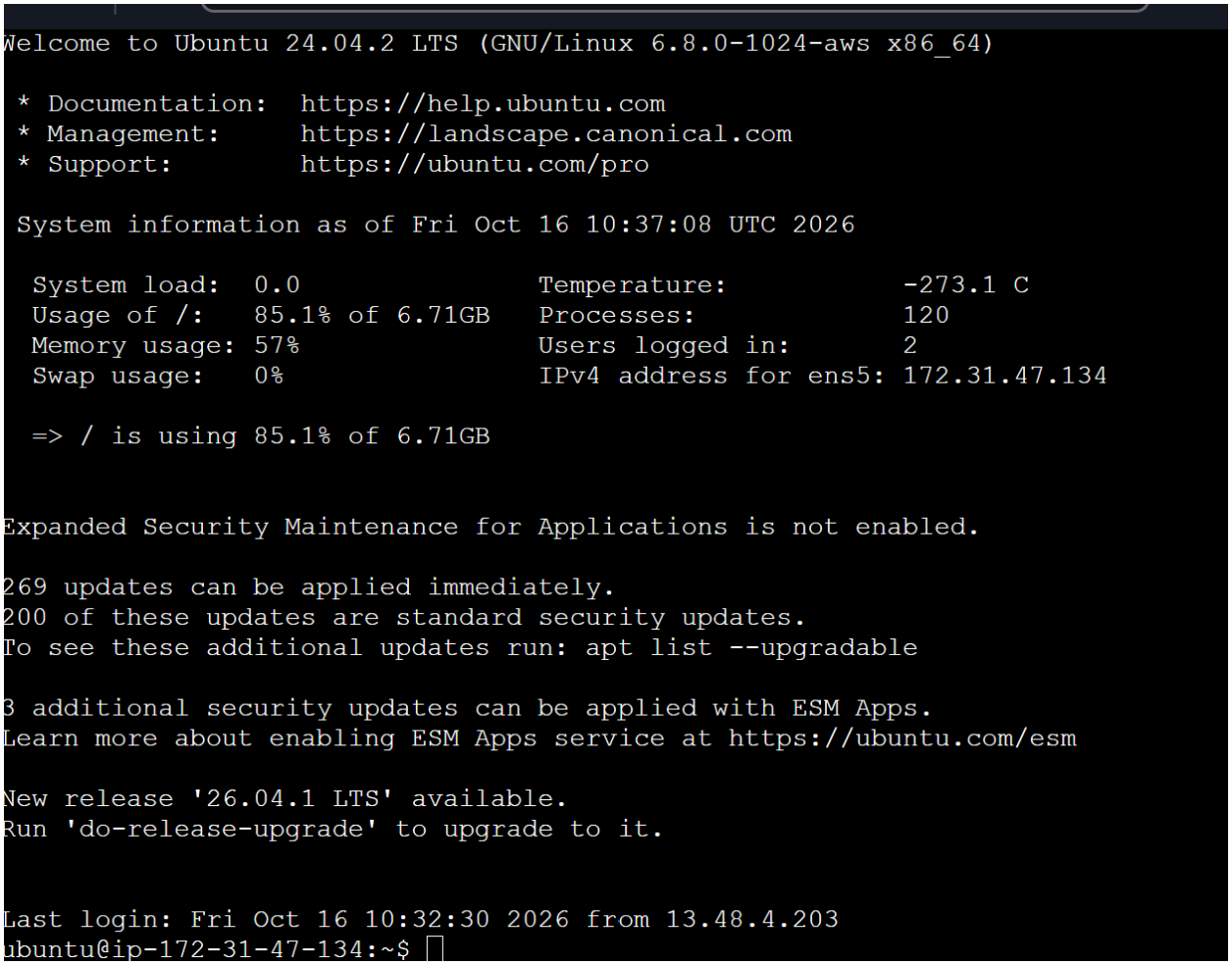
4. Real-time example

A company develops a Spring Boot web application. The DevOps engineer builds the application using Maven, packages it into a Docker image, tests it in a container, and pushes the image to Docker Hub. The same image can then be deployed on other EC2 instances or a Kubernetes cluster.

5. Conclusion

The Java Spring Boot application was successfully built, containerized using Docker, tested on AWS EC2, and pushed to Docker Hub. This process makes the application portable and ready for deployment across compatible environments.





Task 8: Build and Push a Node.js-Based Docker Image

We'll complete your assignment step by step using your EC2 Ubuntu server and Docker Hub account `vineethgupta`.

Important: Your EC2 disk is currently 100% full. We must free some space before building the image, or the build will likely fail again.

Step 1 — Check disk space

Run this command:

df -h /

Then check which folder is using space:

sudo du -xhd1 /var 2>/dev/null | sort -h

Send me the output. We'll fix the disk issue first, then continue with the Dockerfile.

Step 2 — Get the source code

After freeing disk space, run:

cd /tmp

git clone https://github.com/betawins/docker-tasks.git

cd /tmp/docker-tasks/NodeJs_based_source

ls

If the repository already exists, use `cd /tmp/docker-tasks/NodeJs_based_source` instead of cloning it again.

Step 3 — Create the Dockerfile

Open the file:

nano Dockerfile

Paste this Dockerfile:

```
# Stage 1: Build the React application using Node.js

FROM node:20-alpine AS build


WORKDIR /app


COPY package*.json ./

RUN npm ci


COPY . .

RUN npm run build


# Stage 2: Serve the built application using Nginx
```

FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Save with Ctrl+O, press Enter, then exit using Ctrl+X.

Purpose: Node.js builds the application, and Nginx serves the finished website.

Step 4 — Build the image with tag v1

Make sure you are inside the `NodeJs_based_source` directory:

sudo docker build -t vineethgupta/nodejs-app:v1 .

This command may take several minutes. Wait until it completes successfully.

Step 5 — Verify the image

sudo docker images

Look for `vineethgupta/nodejs-app` with tag `v1`.

Step 6 — Run the container

sudo docker run -d --name nodejs-app -p 8084:80 vineethgupta/nodejs-app:v1

Check that it is running:

sudo docker ps

Test the website:

curl -I http://localhost:8084

An HTTP response such as `200 OK` indicates the web server is responding.

Step 7 — Log in to Docker Hub

sudo docker login -u vineethgupta

Enter your Docker Hub access token when prompted. Do not share the token in chat.

Step 8 — Push the image to Docker Hub

sudo docker push vineethgupta/nodejs-app:v1

After success, check your repository here:

Open your Docker Hub repository

Step 9 — Final verification

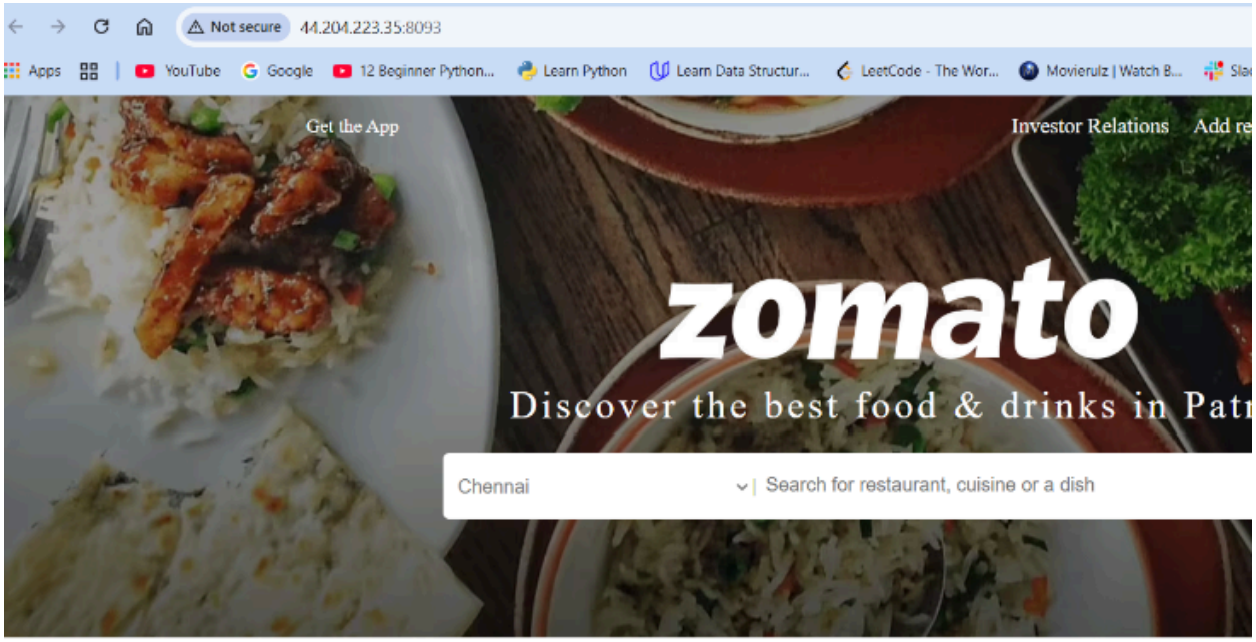
Run:

sudo docker images

sudo docker ps

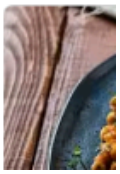
Your expected result:

- Image: `vineethgupta/nodejs-app`
- Tag: `v1`
- Container: `nodejs-app`
- Port mapping: `8084:80`
- Registry: Docker Hub









Task 9.Set Up WordPress with MySQL Using Docker Compose

Objective: Create a `docker-compose.yml` file to run WordPress and a MySQL database in separate Docker containers.

Important: Your EC2 disk was previously 100% full. This task also requires disk space to download the images, so check `df -h /` first if you haven't freed space yet.

Step 1 — Create a project directory

Run these commands on your Ubuntu EC2 server:

```
mkdir -p ~/wordpress-compose

cd ~/wordpress-compose
```

Step 2 — Create the Docker Compose file

nano docker-compose.yml

Paste the following configuration:

```
services:
  db:
    image: mysql:8.0
    container_name: wordpress-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wp_password_123
      MYSQL_ROOT_PASSWORD: root_password_123
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress
    restart: unless-stopped
    depends_on:
      - db
    ports:
      - "8085:80"
```

```
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wp_password_123
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Save with Ctrl+O, press Enter, and exit with Ctrl+X.

Step 3 — Start the containers

First, check that Docker Compose is available:

```
sudo docker compose version
```

If it displays a version, start the services:

```
sudo docker compose up -d
```

Docker will download the WordPress and MySQL images if needed and start both containers.

Step 4 — Verify the containers

```
sudo docker compose ps
```

You should see these two services:

- `wordpress-db` — MySQL database
- `wordpress` — WordPress web application

Check the WordPress response:

curl -I http://localhost:8085

Step 5 — Open WordPress

Open this address in your browser:

http://13.60.47.140:8085

If it doesn't open, check that your EC2 security group's inbound rules allow TCP port 8085 from your IP address.

WordPress should display its initial setup page, where you can choose the site language and create the administrator account.

How it works

Browser

Port 8085

WordPress container

Web application · Port 80

MySQL container

Stores WordPress data

The Compose file connects the containers, configures database credentials, and creates persistent volumes so data survives container recreation.

Note: These passwords are for a practice lab only. Use strong, private passwords for real deployments, and don't expose the MySQL port publicly.

Useful commands

Task	Command
View running services	`sudo docker compose ps`

View logs	`sudo docker compose logs`
Stop services	`sudo docker compose stop`
Stop and remove containers	`sudo docker compose down`

Expected result: WordPress and MySQL run as separate containers managed together by Docker Compose.

